

Automated Deterministic Auction Design with Objective Decomposition

Zhijian Duan*

School of Computer Science
Peking University
Beijing, Beijing, China
zjduan@pku.edu.cn

Haoran Sun*

School of Computer Science
Peking University
Beijing, Beijing, China
sunhaoran0301@stu.pku.edu.cn

Yichong Xia†

School of Computer Science
Peking University
Beijing, Beijing, China
xiayc@stu.pku.edu.cn

Siqiang Wang†

Foster School of Business
University of Washington
Seattle, Washington, United States
siqiauw2@uw.edu

Zhilin Zhang

Alibaba Group
Beijing, Beijing, China
zhangzhilin.pt@alibaba-inc.com

Chuan Yu

Alibaba Group
Beijing, Beijing, China
yuchuan.yc@alibaba-inc.com

Jian Xu

Alibaba Group
Beijing, Beijing, China
xiyu.xj@alibaba-inc.com

Xiaotie Deng

Center on Frontiers of Computing
Studies, School of Computer Science
Peking University
Beijing, Beijing, China
xiaotie@pku.edu.cn

Abstract

Identifying high-revenue mechanisms that are both dominant strategy incentive compatible (DSIC) and individually rational (IR) is a fundamental challenge in auction design. While theoretical approaches have encountered bottlenecks in multi-item combinatorial auctions, there has been much empirical progress in the automated design of such mechanisms using machine learning. However, existing research primarily focuses on randomized auctions, with less attention given to more practical deterministic auctions. Therefore, in this paper, we introduce OD-VVCA, an objective decomposition approach for automated designing revenue-maximizing deterministic Virtual Valuations Combinatorial Auctions (VVCAs), which are inherently DSIC and IR. We use a parallelizable dynamic programming algorithm to compute the allocation and revenue outcomes of a VVCA efficiently. We then decompose the revenue objective function into continuous and piecewise-constant discontinuous components, optimizing each using distinct methods. Extensive experiments show that OD-VVCA achieves high revenue in multi-item auctions, especially in large-scale settings where it outperforms both randomized and deterministic baselines, indicating its efficacy and scalability.

*Both authors contributed equally to this research.

†Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792377>

CCS Concepts

• **Theory of computation** → **Algorithmic game theory and mechanism design; Algorithmic mechanism design; Computational pricing and auctions.**

Keywords

Automated Mechanism Design, Differentiable Economics, Auction Design, Machine Learning

ACM Reference Format:

Zhijian Duan, Haoran Sun, Yichong Xia, Siqiang Wang, Zhilin Zhang, Chuan Yu, Jian Xu, and Xiaotie Deng. 2026. Automated Deterministic Auction Design with Objective Decomposition. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792377>

1 Introduction

Finding high-revenue mechanisms that are both *dominant strategy incentive compatible* (DSIC) and *individually rational* (IR) is a fundamental challenge in auction design. A DSIC and IR mechanism incentivizes bidders to truthfully report their valuations while ensuring they receive non-negative utilities. Although Myerson [24] characterized the optimal design for single-item auctions, theoretical approaches have faced significant challenges in multi-item combinatorial auctions [14]. Consequently, there has been much progress in automated auction design, which formulates a constrained optimization problem and empirically seeks high-revenue mechanisms using machine learning [8, 13, 32, 33].

Existing research on the automated design of multi-item auctions focuses primarily on randomized auctions, which produce randomized allocation outcomes [7, 10–13, 17, 25–27], with deterministic auctions receiving less attention. However, deterministic auctions

offer practical advantages over random ones in real-world applications because deterministic allocations are more interpretable and easier to implement, leading to widespread adoption in online advertising [15, 20, 23]. Motivated by these practical considerations, this paper investigates the automated design of deterministic mechanisms for multi-item combinatorial auctions.

Existing approaches to automated deterministic auction design for multi-item combinatorial auctions predominantly rely on Affine Maximizer Auctions (AMAs) [22, 30, 32]. AMAs extend Vickrey-Clarke-Groves (VCG) auctions [35] by assigning weights to each bidder and enhancing candidate allocations to achieve higher revenue while preserving DSIC and IR. However, these approaches face two significant challenges. First, *scalability* poses a critical obstacle. AMAs enhance every candidate allocation, resulting in $O((n+1)^m)$ boost parameters for n bidders and m items, which grows exponentially. Furthermore, computing the optimal allocation in an AMA requires finding the winning allocation that maximizes the affine welfare, a task that is NP-Hard [6, 31]. Second, the revenue function of AMAs is *non-differentiable* with respect to the AMA parameters. This issue, which we elaborate on in Section 4.3, arises because revenue computation entails identifying the winning allocation from among all $(n+1)^m$ possible deterministic allocations.

To address the scalability and non-differentiability issues of AMA-based automated deterministic auction design, we introduce OD-VVCA, an objective decomposition approach for designing Virtual Valuations Combinatorial Auctions (VVCAs) [22, 32]. Firstly, we constrain the mechanism to be deterministic VVCAs, a subset of deterministic AMAs that boost each bidder-bundle pair, reducing the parameter space from $O((n+1)^m)$ to $O(n \times 2^m)$. Secondly, since determining the winning allocations of VVCAs remains NP-Hard [22], we use a parallelizable dynamic programming (DP) algorithm to solve the problem relatively efficiently. Thirdly, we decompose the objective function, i.e., the revenue of the VVCA, into continuous and discontinuous components. The continuous component is differentiable almost everywhere, allowing us to take its derivative directly. The discontinuous component is a piecewise constant function, and we use Monte Carlo estimation to approximate the gradient of its Gaussian smoothing. Finally, by integrating the gradients from both components, we optimize the VVCA parameters using gradient ascent.

Extensive experiments show that OD-VVCA can achieve higher revenue than deterministic baselines and even randomized AMA-based approaches, particularly in large-scale asymmetric auctions. Additionally, we conduct an ablation study and case studies to showcase the significant benefits that the objective decomposition brings to OD-VVCA in terms of revenue and optimization. All these experimental results indicate the effectiveness of our approach.

2 Related Work

Research in automated auction design can be broadly categorized into three threads. The first thread, initiated by RegretNet [13], utilizes ex-post regret as a metric to quantify the extent of DSIC violation. Building upon this concept, subsequent works [7, 9, 12, 17, 25–27] represent auction mechanisms through neural networks. These

models aim to achieve near-optimal and approximate DSIC solutions through adversarial training. The advantages of these regret-based methods lie in their ability to attain high revenue, considering a broad class of auction mechanisms. Furthermore, this methodology is generalizable and applicable to various other mechanism design problems, including multi-facility location [16], two-sided matching [28], and data markets [29]. However, the regret-based methods are not guaranteed to be DSIC, and computing the regret term can be time-consuming. Moreover, all the regret-based methods are designed for randomized auctions.

The second thread is based on AMAs [30], a weighted variation of the VCG mechanism. AMAs assign weights to each bidder and boost each candidate allocation, enabling them to achieve higher revenue than VCG while maintaining DSIC. This method is particularly well-suited for scenarios with limited candidate allocations [20]. Furthermore, the sample complexity of AMA has been characterized by Balcan et al. [2, 3, 4]. However, standard AMAs face scalability challenges in multi-item auctions with n bidders and m items, as there are $(n+1)^m$ deterministic candidate allocations. A solution is to restrict the candidate set to size $s \ll (n+1)^m$ and make it learnable [10, 11], resulting in randomized candidate allocations. Still, the effective size grows with the number of bidders and items. Another strategy is to constrain the mechanism to structured subsets such as VVCAs [21, 22, 32], λ -auctions [18], mixed bundling auctions [34], and bundling-boosted auctions [1], among which VVCAs form the most expressive class. We therefore focus on VVCAs. However, VVCAs suffer from revenue non-differentiability. Existing VVCA-based methods [22, 32] use heuristic gradients computed under fixed winning allocations. In contrast, we estimate gradients via objective decomposition, explicitly capturing the dependence of the winning allocation on the VVCA parameters.

The third thread is led by GemNet [36], which constructs randomized menus (candidate allocations) for each bidder. Based on learnable bidder-level candidate allocations, they design a mechanism class that is not only DSIC and IR but also exhibits stronger representativeness than AMAs. However, GemNet requires additional effort to ensure the feasibility of the allocation results (i.e., no item is over-allocated). These non-trivial and time-consuming efforts hinder its scalability, which we will discuss in our comparison experiments.

3 Preliminaries

Sealed-Bid Auction. A sealed-bid auction involves n bidders denoted as $[n] = \{1, 2, \dots, n\}$ and m items denoted as $[m] = \{1, 2, \dots, m\}$. Each bidder i assigns a valuation $v_i(S)$ to every bundle of items $S \subseteq [m]$ and submits bids for each bundle as $\mathbf{b}_i \in \mathbb{R}^{2^m}$. We assume $v_i(\emptyset) = 0$ for all bidders. In the additive setting, $v_i(S) = \sum_{j \in S} v_i(\{j\}) = \sum_{j \in S} v_{ij}$. The valuation profile $V = (v_1, v_2, \dots, v_n)$ is generated from a distribution D . The auctioneer lacks knowledge of the true valuation profile V but can observe the bidding profile $B = (\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n)$.

Auction Mechanism. An auction mechanism (g, p) consists of an allocation rule g and a payment rule p . Given the bids B , $g_i(B)$ computes the allocation result for bidder i , which can be a bundle of items or a probability distribution over all the bundles. The payment $p_i(B) \geq 0$ computes the price that bidder i needs to pay.

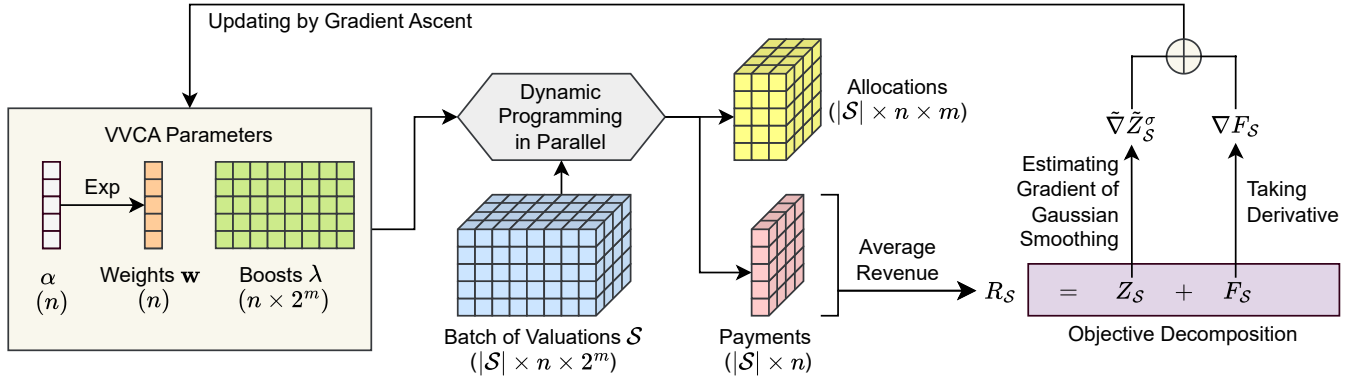


Figure 1: Overview of OD-VVCA. We begin by setting up the VVCA parameters $\mathbf{w} = e^\alpha \in \mathbb{R}_+^n$ and $\lambda \in \mathbb{R}^{n \times 2^m}$. A parallelizable dynamic programming algorithm is employed to compute the allocations and payments for a batch of valuations. Afterward, the average revenue objective is decomposed into two components, and their gradients are estimated or computed directly. By integrating these gradients, we update the VVCA parameters through gradient ascent.

Each bidder aims to maximize her utility, defined as $u_i(\mathbf{v}_i, B) := v_i(g_i(B)) - p_i(B)$. Bidders may misreport their valuations to gain an advantage. Such strategic behavior among bidders could make the auction result hard to predict. Therefore, we require the auction mechanism to be *dominant strategy incentive compatible (DSIC)*, meaning that for each bidder $i \in [n]$, reporting her true valuation is her optimal strategy regardless of how others report. Formally, a DSIC mechanism satisfies that, for any bidder $i \in [n]$ and any bids of others $B_{-i} = (b_1, \dots, b_{i-1}, b_{i+1}, \dots, b_n)$, we have

$$u_i(\mathbf{v}_i, (\mathbf{v}_i, B_{-i})) \geq u_i(\mathbf{v}_i, (b_i, B_{-i})), \quad \forall i, \mathbf{v}_i, B_{-i}, b_i. \quad (\text{DSIC})$$

Furthermore, the auction mechanism needs to be *individually rational (IR)*, ensuring that truthful bidding results in a non-negative utility for each bidder. Formally,

$$u_i(\mathbf{v}_i, (\mathbf{v}_i, B_{-i})) \geq 0, \quad \forall i, \mathbf{v}_i, B_{-i}. \quad (\text{IR})$$

Affine Maximizer Auction. AMAs [30] is a generalized version of VCG auctions [35] and inherently ensures DSIC and IR. A (deterministic) AMA contains positive weights $w_i \in \mathbb{R}_+$ for each bidder $i \in [n]$ and boosts $\lambda(A) \in \mathbb{R}$ for each candidate allocation $A \in \mathcal{A}$, where $\mathcal{A}$ is the set of all the $(n+1)^m$ deterministic allocations. Given bids B , an AMA of parameters $(\mathbf{w}, \lambda)$ selects the allocation A^* that maximizes the affine welfare (with an arbitrary tie-breaking rule):

$$A^* = g(B; \mathbf{w}, \lambda) := \operatorname{argmax}_{A \in \mathcal{A}} \sum_{i=1}^n w_i b_i(A) + \lambda(A), \quad (1)$$

and each bidder i pays for her normalized negative affine welfare impact on other bidders:

$$p_i(B; \mathbf{w}, \lambda) = \frac{1}{w_i} \max_{A \in \mathcal{A}} \left(\sum_{j \neq i} w_j b_j(A) + \lambda(A) \right) - \frac{1}{w_i} \left(\sum_{j \neq i} w_j b_j(A^*) + \lambda(A^*) \right). \quad (2)$$

Virtual Valuations Combinatorial Auction. VVCAs [22] is a subset of AMAs, distinguishing itself by decomposing the boost variable of AMAs into n parts, one for each bidder:

$$\lambda(A) = \sum_{i=1}^n \lambda_i(A_i). \quad (3)$$

This decomposition reduces parameters from $O((n+1)^m)$ to $O(n \times 2^m)$, aligning with the same order as the input valuation V .

4 Methodology

In this section, we outline the methodology of OD-VVCA, as depicted in Figure 1. We begin by formalizing our problem of automated mechanism design for deterministic VVCAs. Next, we introduce the DP algorithm for computing the auction results of VVCAs and show that it can be parallel implemented in practice. Finally, we explain how we optimize the VVCA parameters using objective decomposition.

4.1 Problem Formulation

We aim to discover a high-revenue deterministic auction mechanism that satisfies both DSIC and IR for a sealed-bid auction with n bidders and m items. Since there is currently no known characterization of a DSIC multi-item combinatorial auction [14], a common strategy is to limit the auction class to AMAs [10, 11, 22, 32]. AMAs are inherently DSIC and IR, encompassing a broad class of mechanisms [19]. However, the boost parameters λ are defined across the entire candidate allocations, resulting in $O((n+1)^m)$ parameters. To address the scalability challenges stemming from the parameter space of AMAs, we further constrain the auction class to be (deterministic) VVCAs. Based on this constraint, we can formalize the auction design problem as an optimization problem:

$$\max_{\mathbf{w} \in \mathbb{R}_+^n, \lambda \in \mathbb{R}^{n \times 2^m}} R_D(\mathbf{w}, \lambda) := \mathbb{E}_{V \sim D} \left[\sum_{i=1}^n p_i(V; \mathbf{w}, \lambda) \right]. \quad (4)$$

We define $R(V, \mathbf{w}, \boldsymbol{\lambda}) := \sum_{i=1}^n p_i(V; \mathbf{w}, \boldsymbol{\lambda})$ as the revenue of valuation profile V under VVCA parameters $\mathbf{w}$ and $\boldsymbol{\lambda}$. $R(V, \mathbf{w}, \boldsymbol{\lambda})$ can be decomposed into two components, the discontinuous part $Z(V, \mathbf{w}, \boldsymbol{\lambda})$ and the continuous part $F(V, \mathbf{w}, \boldsymbol{\lambda})$. Formally,

$$R(V, \mathbf{w}, \boldsymbol{\lambda}) = Z(V, \mathbf{w}, \boldsymbol{\lambda}) + F(V, \mathbf{w}, \boldsymbol{\lambda}), \quad (5)$$

where

$$Z(V, \mathbf{w}, \boldsymbol{\lambda}) := \sum_{i=1}^n v_i(g(V; \mathbf{w}, \boldsymbol{\lambda})), \quad (6)$$

and

$$F(V, \mathbf{w}, \boldsymbol{\lambda}) := \sum_{i=1}^n \frac{1}{w_i} \max_{A \in \mathcal{A}} \left(\sum_{j \neq i} w_j v_j(A) + \sum_{k=1}^n \lambda_k(A_k) \right) - \sum_{i=1}^n \frac{1}{w_i} \max_{A \in \mathcal{A}} \left(\sum_{j=1}^n w_j v_j(A) + \lambda_j(A_j) \right) \quad (7)$$

are the two components of $R(V, \mathbf{w}, \boldsymbol{\lambda})$. Although both $F(V, \mathbf{w}, \boldsymbol{\lambda})$ and $Z(V, \mathbf{w}, \boldsymbol{\lambda})$ are almost everywhere differentiable, $Z(V, \mathbf{w}, \boldsymbol{\lambda})$ is a piecewise constant function and thus discontinuous. The discontinuities occur when the allocation result $g(V; \mathbf{w}, \boldsymbol{\lambda})$ changes with $\mathbf{w}$ and $\boldsymbol{\lambda}$. In contrast, $F(V, \mathbf{w}, \boldsymbol{\lambda})$ is continuous, making it easier to be optimized through the gradient method.

4.2 Winning Allocation Determination

As shown in Equation (5), computing $R(V, \mathbf{w}, \boldsymbol{\lambda})$ requires finding the winning allocation $g(V; \mathbf{w}, \boldsymbol{\lambda})$ defined in Equation (1), along with n allocations for each bidder $i \in [n]$ that solve the optimization problem $\max_{A \in \mathcal{A}} \left(\sum_{j \neq i} w_j v_j(A) + \sum_{k=1}^n \lambda_k(A_k) \right)$, which represents the winning allocation for the valuation profile $(0, V_{-i})$. Thus, we need to develop an algorithm to compute the winning allocation for an arbitrary valuation profile. However, the winning allocation determination of a VVCA is NP-Hard [6, 31]. To address this challenge as efficiently as possible, we use a parallelizable dynamic programming (DP) algorithm for computing the winning allocation of VVCAs.

In the dynamic program (DP) for determining the winning allocation, we define $\text{MAW}(k, S)$ as the Maximum Affine Welfare when item bundle $S \in 2^m$ is allocated to the first k bidders, and we use $\text{AB}(k, S)$ to record the corresponding Allocated Bundle for bidder k . We initialize $\text{MAW}(1, S) = w_1 v_1(S) + \lambda_1(S)$ and $\text{AB}(1, S) = S$. Then, we iteratively compute $\text{MAW}(k, S)$ and $\text{AB}(k, S)$ for $k = 2, 3, \dots, n$ and $S \in 2^m$. The computation of $\text{MAW}(k, S)$ and $\text{AB}(k, S)$ involves enumerating all subsets $\mathcal{B} \subseteq S$ as potential bundles allocated to bidder k , as follows:

$$\begin{aligned} \text{MAW}(k, S) &= \max_{\mathcal{B} \subseteq S} \text{MAW}(k-1, S \setminus \mathcal{B}) + w_k v_k(\mathcal{B}) + \lambda_k(\mathcal{B}), \\ \text{AB}(k, S) &= \arg\max_{\mathcal{B} \subseteq S} \text{MAW}(k-1, S \setminus \mathcal{B}) + w_k v_k(\mathcal{B}) + \lambda_k(\mathcal{B}). \end{aligned} \quad (8)$$

Finally, to get the winning allocation result, we define the set of all allocated items in the affine welfare-maximizing allocation as $S_n := \arg\max_{S \subseteq [m]} \text{MAW}(n, S)$. Based on S_n , We iteratively determine the allocated bundle $\text{AB}(k, S_k)$ for bidder $k = n, n-1, \dots, 1$, updating $S_{k-1} := S_k - \text{AB}(k, S_k)$.

We further show that the computation of Equation (8) can be accelerated using vector operations: For any item bundle $S \subseteq [m]$, let $\vec{\mathcal{B}}_S := (\mathcal{B})_{\mathcal{B} \subseteq S}$ denote the $2^{|S|}$ -dimensional vector of all

Algorithm 1 Winning Allocation Determination by Dynamic Programming

```

1: Input: Valuation  $V$ , VVCA parameters  $\mathbf{w} \in \mathbb{R}_+^n$  and  $\boldsymbol{\lambda} \in \mathbb{R}^{n2^m}$ .
2: Output: Winning allocation  $A^*$ .
3: for  $S \subseteq [m]$  do
4:   Initialize  $\text{MAW}(1, S) \leftarrow w_1 v_1(S) + \lambda_1(S)$ .
5:   Initialize  $\text{AB}(1, S) \leftarrow S$ .
6:   Initialize  $\vec{\mathcal{B}}_S \leftarrow (\mathcal{B})_{\mathcal{B} \subseteq S}$  as the vector of all subsets of  $S$ .
7: end for
8: for  $k = 2$  to  $n$  do
9:   for  $S \subseteq [m]$  do
10:     $\text{Int}(k, S, \vec{\mathcal{B}}_S) \leftarrow \text{MAW}(k-1, S - \vec{\mathcal{B}}_S) + w_k v_k(\vec{\mathcal{B}}_S) + \lambda_k(\vec{\mathcal{B}}_S)$ 
11:     $\text{MAW}(k, S) \leftarrow \max_{\mathcal{B} \in \vec{\mathcal{B}}_S} \text{Int}(k, S, \mathcal{B})$ 
12:     $\text{AB}(k, S) \leftarrow \arg\max_{\mathcal{B} \in \vec{\mathcal{B}}_S} \text{Int}(k, S, \mathcal{B})$ 
13:   end for
14: end for
15:  $S_n \leftarrow \arg\max_{S \subseteq [m]} \text{MAW}(n, S)$ .
16: for  $k = n$  to  $1$  do
17:    $A_k^* \leftarrow \text{AB}(k, S_k)$ , the allocated bundle of bidder  $k$ .
18:    $S_{k-1} \leftarrow S_k - A_k^*$ 
19: end for
20: return Winning Allocation  $A^* = (A_1^*, A_2^*, \dots, A_n^*)$ 

```

subsets of S . Using this, we compute the intermediate result vector $\text{Int}(k, S, \vec{\mathcal{B}}_S) = \text{MAW}(k-1, S \setminus \vec{\mathcal{B}}_S) + w_k v_k(\vec{\mathcal{B}}_S) + \lambda_k(\vec{\mathcal{B}}_S)$, which captures the result of assigning item bundle S to the first k bidders. Subsequently, we compute $\text{MAW}(k, S)$ and $\text{AB}(k, S)$ by identifying the maximum element of the vector $\text{Int}(k, S, \vec{\mathcal{B}}_S)$, i.e., $\text{MAW}(k, S) = \max_{\mathcal{B} \in \vec{\mathcal{B}}_S} \text{Int}(k, S, \mathcal{B})$ and $\text{AB}(k, S) = \arg\max_{\mathcal{B} \in \vec{\mathcal{B}}_S} \text{Int}(k, S, \mathcal{B})$, where all operations are vector-based.

In summary, we provide the pseudocode of the parallel DP implementation in Algorithm 1. During the training process, we often get a minibatch of the valuations $\mathcal{S} = \{V^1, V^2, \dots, V^S\}$ at the same time. Since we have shown that the DP computation for a single valuation involves vector operations, we can compute all the DP computations through matrix operations. Thus, the computation for all valuations can be executed using matrix operations.

We also prove in Appendix A that the time complexity and space complexity of the DP algorithm are $O(n3^m)$ and $O(n2^m)$, respectively. Notably, the input valuation profile is already $O(n2^m)$, demonstrating the efficiency of our dynamic programming algorithm. Based on the DP algorithm, we can efficiently compute $R(V, \mathbf{w}, \boldsymbol{\lambda})$ as described in Equation (5). The computation takes $n+1$ DPs: one to solve the winning allocation for valuation profile V , and n additional DPs to solve for the valuation profile $(0, V_{-i})$ for each bidder $i \in [n]$.

4.3 Optimization

To optimize the VVCA parameters $\mathbf{w}$ and $\boldsymbol{\lambda}$ in Equation (4), we first introduce $\boldsymbol{\alpha} \in \mathbb{R}^n$ as the logarithm of $\mathbf{w}$ (such that $\mathbf{w} = e^{\boldsymbol{\alpha}} \in \mathbb{R}_+^n$) and optimize $\boldsymbol{\alpha}$ instead to handle the positive range of $\mathbf{w}$. Then, we

optimize the empirical revenue, which is defined as:

$$\begin{aligned} R_S(\mathbf{w} = e^\alpha, \lambda) &:= \frac{1}{|S|} \sum_{V \in S} R(V, e^\alpha, \lambda) \\ &= \underbrace{\frac{1}{|S|} \sum_{V \in S} Z(V, e^\alpha, \lambda)}_{Z_S(e^\alpha, \lambda)} + \underbrace{\frac{1}{|S|} \sum_{V \in S} F(V, e^\alpha, \lambda)}_{F_S(e^\alpha, \lambda)}, \end{aligned} \quad (9)$$

where S is a dataset of valuations sampled i.i.d. from D . However, $R_S(e^\alpha, \lambda)$ is non-differentiable with respect to α and λ , since the computation of $R(V, e^\alpha, \lambda)$ involves finding the winning allocation from all deterministic candidate allocations and computing the social welfare of the winning allocation (i.e., $Z(V, e^\alpha, \lambda)$ in Equation (5)). Therefore, we decompose it into two parts: $F_S(e^\alpha, \lambda)$ and $Z_S(e^\alpha, \lambda)$, and optimize them separately.

The optimization of $F_S(e^\alpha, \lambda)$ is straightforward since it is a continuous and piece-wise non-constant linear function with respect to e^α and λ . Thus, we can easily optimize $F_S(e^\alpha, \lambda)$ through its gradient $\nabla F_S(e^\alpha, \lambda)$, where the gradient of the maximum function can be estimated similarly to the ReLU function.

However, optimizing $Z_S(e^\alpha, \lambda)$ is more challenging. This is because $Z_S(e^\alpha, \lambda)$ is a discontinuous, piece-wise constant function. The discontinuities occur when the allocation result $g(V; \mathbf{w}, \lambda)$ changes with $\mathbf{w}$ and λ . Its derivatives at differentiable points are always 0, and it undergoes sudden changes at the discontinuous points, making it not feasible to optimize $Z_S(e^\alpha, \lambda)$ using its own gradient. To address this issue, similar to Bichler et al. [5], we optimize $Z_S(e^\alpha, \lambda)$ using Gaussian smoothing techniques. Specifically, we optimize the Gaussian smoothing approximation of $Z_S(e^\alpha, \lambda)$, which is defined as:

$$\tilde{Z}_S^\sigma(e^\alpha, \lambda) := \mathbb{E}_{\epsilon, \delta} [Z_S(e^{\alpha+\sigma\epsilon}, \lambda + \sigma\delta)],$$

where $\epsilon \sim N(0, 1)^n$, $\delta \sim N(0, 1)^{n \times 2^m}$, $\sigma > 0$ and $N(0, 1)$ is the standard normal distribution. Clearly, as $\sigma \rightarrow 0$, $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$ approaches $Z_S(e^\alpha, \lambda)$. As for the differentiability property of $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$, we have the following proposition:

Proposition 4.1. $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$ is continuous and differentiable with derivatives

$$\begin{aligned} \nabla_\alpha \tilde{Z}_S^\sigma(e^\alpha, \lambda) &= \mathbb{E}_{\epsilon \sim N(0, 1)^n, \delta \sim N(0, 1)^{n \times 2^m}} \frac{\Delta_\epsilon^\delta}{\sigma} \epsilon, \\ \nabla_\lambda \tilde{Z}_S^\sigma(e^\alpha, \lambda) &= \mathbb{E}_{\epsilon \sim N(0, 1)^n, \delta \sim N(0, 1)^{n \times 2^m}} \frac{\Delta_\epsilon^\delta}{\sigma} \delta, \end{aligned}$$

where we define $\Delta_\epsilon^\delta := Z_S(e^{\alpha+\sigma\epsilon}, \lambda + \sigma\delta) - Z_S(e^\alpha, \lambda)$.

According to Proposition 4.1, we can unbiasedly estimate the gradient of $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$ by Monte Carlo sampling, which is done by computing the average of numerical differentiations in n_r random directions generated by Gaussian sampling:

$$\begin{aligned} \nabla_\alpha \tilde{Z}_S(e^\alpha, \lambda) &\approx \tilde{\nabla}_\alpha \tilde{Z}_S(e^\alpha, \lambda) \\ &= \frac{1}{n_r} \sum_{i=1}^{n_r} \frac{1}{\sigma} (Z_S(e^{\alpha+\sigma\epsilon_i}, \lambda + \sigma\delta_i) - Z_S(e^\alpha, \lambda)) \epsilon_i, \\ \nabla_\lambda \tilde{Z}_S(e^\alpha, \lambda) &\approx \tilde{\nabla}_\lambda \tilde{Z}_S(e^\alpha, \lambda) \\ &= \frac{1}{n_r} \sum_{i=1}^{n_r} \frac{1}{\sigma} (Z_S(e^{\alpha+\sigma\epsilon_i}, \lambda + \sigma\delta_i) - Z_S(e^\alpha, \lambda)) \delta_i, \end{aligned}$$

where $\epsilon_i \sim N(0, 1)^n$, and $\delta_i \sim N(0, 1)^{n \times 2^m}$. It takes totally n_r additional DPs to get the estimated gradient $\tilde{\nabla} Z_S(e^\alpha, \lambda)$, based on which we can optimize $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$ through gradient ascent. This approach offers an approximate method for optimizing $Z_S(e^\alpha, \lambda)$.

It is also feasible to optimize $R_S(e^\alpha, \lambda)$ using the same Gaussian smoothing technique as $Z_S(e^\alpha, \lambda)$. However, this approach would require $(n+1)n_r$ additional DPs to compute the estimated gradient. In comparison, our objective decomposition reduces the number of additional DPs to n_r , enhancing efficiency significantly.

5 Experiments

In this section, we present empirical experiments to evaluate the effectiveness of OD-VVCA.¹ These experiments are conducted on a Linux machine equipped with 4 NVIDIA Graphics Processing Units (GPUs), each with a memory size of 11GB. Each result is averaged across 5 distinct runs. The standard deviation of OD-VVCA across all these runs is below 1%.

5.1 Setup

Auction Settings. We consider a variety of valuation distributions, including both symmetric and asymmetric, as well as additive and combinatorial types. They are listed as follows:

- (A) (Symmetric Uniform) For every bidder i and item j , the valuation v_{ij} is sampled from $U[0, 1]$. The valuation is additive, that is, for all item bundle S and bidder i , $v_i(S) = \sum_{j \in S} v_{ij}$. This setting is widely used in previous research [10, 13, 32].
- (B) (Asymmetric Uniform) For all bidder i and item j , the valuation v_{ij} is sampled from $U[0, i]$. The valuation is additive but asymmetric for bidders.
- (C) (Lognormal) For all bidder i and item j , the valuation v_{ij} is sampled from $\text{Lognormal}(0, 1/i^2)$. The valuation is additive but asymmetric for bidders.
- (D) (Combinatorial) For all bidder i and item j , the valuation v_{ij} is sampled from $U[1, 2]$. For item bundle $S \subseteq [m]$ and bidder i , the valuation $v_i(S) = \sum_{j \in S} v_{ij} + \epsilon_{iS}$, where ϵ_{iS} is sampled from $U[-|S|/2, |S|/2]$. This setting is also used in Sandholm and Likhodedov [32].

Baselines. We compare OD-VVCA against both randomized and deterministic methods. Note that we ignore the regret-based methods [13] since they are not strictly IC and thus not directly comparable to OD-VVCA. The randomized methods include:

- (1) *Lottery AMA* [10], a randomized AMA-based approach that directly sets the candidate allocations, bidder weights, and boost variables as all the learnable weights.
- (2) *AMenuNet* [11], similar to Lottery AMA, but uses a transformer-based architecture to optimize the parameters of AMA, hence more powerful and applicable to general contextual auctions.
- (3) *GemNet* [36], which employs a DSIC and IR mechanism class based on learnable bidder-level menus (candidate allocations). However, it requires solving a mixed-integer linear program to ensure allocation feasibility (i.e., no item is over-allocated), which is computationally expensive especially in large-scale auctions.

The deterministic methods include:

¹The implementation is available at <https://github.com/Haoran0301/OD-VVCA>.

Table 1: The revenue results in both small and large settings. For each setting, the highest revenue among all methods is **bolded, and the highest revenue among all deterministic methods (the methods except for Lottery AMA and AMenuNet) is underlined. We do not report the results of Item-Myerson, Lottery AMA, AMenuNet, and GemNet for setting (D) because Item-Myerson is not DSIC in a non-additive scenario, and Lottery AMA, AMenuNet, and GemNet are not applicable for combinatorial valuations.**

(a) Revenue results in small-scale settings. Some results for GemNet are not provided because it fails to compute a feasible allocation within a reasonable time due to the overly complex constraints of its mixed-integer LP.

Type	Method	Symmetric					Asymmetric				
		2×2(A)	2×5(A)	3×10(A)	2×2(D)	3×10(D)	5×3(B)	3×10(B)	2×5(C)	5×3(C)	3×10(C)
R	Lottery AMA	0.8680	2.2354	5.3450	N/A	N/A	6.5904	11.9814	5.7001	4.0195	12.9863
	AMenuNet	0.8618	2.2768	5.5986	N/A	N/A	6.7743	12.3419	5.6512	4.1919	13.6094
	GemNet	0.8780	2.3100	—	N/A	N/A	6.7320	—	5.5330	4.8343	—
D	VCG	0.6678	1.6691	5.0032	2.4576	15.6305	6.0470	8.9098	3.8711	3.7249	10.6495
	Item-Myerson	<u>0.8330</u>	2.0755	5.3141	N/A	N/A	5.3909	8.9110	4.4380	3.6918	9.3740
	BBBVCA	0.7781	2.2576	5.7647	2.6111	16.3504	6.6118	10.5134	4.8174	4.1879	11.4605
	FO-VVCA	0.7836	<u>2.2638</u>	5.7876	2.6205	16.3786	6.8783	11.8093	5.5536	4.1906	13.0144
	OD-VVCA	0.8284	2.2632	<u>5.8230</u>	2.6802	16.3675	7.0344	12.5497	<u>5.6682</u>	<u>4.3289</u>	13.6223

(b) Revenue results in large-scale settings. GemNet is excluded as its overly complex constraints make it impractical to compute a feasible allocation in large-scale settings.

Type	Method	Symmetric				Asymmetric					
		5×10(A)	10×5(A)	5×10(D)	10×5(D)	5×10(B)	10×5(B)	30×5(B)	5×10(C)	10×5(C)	30×5(C)
R	Lottery AMA	5.5435	3.0210	N/A	N/A	21.4092	24.3684	74.1259	13.0961	6.1878	4.8812
	AMenuNet	6.5210	3.8156	N/A	N/A	21.7912	27.4230	88.2373	13.6266	7.0347	6.1055
D	VCG	6.6690	4.0887	17.2580	9.4405	20.1567	26.7979	105.4290	12.4163	6.5472	6.6054
	Item-Myerson	6.7132	4.0868	N/A	N/A	17.9697	24.7287	102.1878	12.3060	7.4868	7.6173
	BBBVCA	6.9765	4.1038	17.4657	9.4356	22.4459	28.2620	107.5630	14.4585	7.4899	7.4989
	FO-VVCA	6.9904	4.1107	17.5026	9.4397	23.1474	28.4363	107.8873	14.6641	7.2678	7.0965
	OD-VVCA	6.9848	4.0830	17.4614	9.4388	24.4301	28.9711	108.2780	14.8071	7.5882	7.6446

- (1) VCG [35], which is the most classical special case of VVCA.
- (2) Item-Myerson [24], which independently applies Myerson auction with respect to each item.
- (3) BBBVCA [22, 32], the pioneering work of optimizing VVCA parameters, which minimizes the revenue loss for each bidder-bundle by gradient descent. We use the same DP algorithm as OD-VVCA to compute the winning allocation.
- (4) FO-VVCA, an ablated version of OD-VVCA which only optimizes $F_S(e^\alpha, \lambda)$.

Hyperparameters and Implementation. For OD-VVCA, we select the number of random directions n_r from $\{8, 16, 32\}$ and the standard deviation σ from $\{0.01, 0.003, 0.001\}$. The training starts at classic VCG, i.e., $w = 1$ and $\lambda = 0$. For Lottery AMA and AMenuNet, since the candidate allocation set cannot be arbitrarily large due to the constraint of computational resources, we select the candidate size from $\{32, 128, 1024, 4096\}$ to balance their performance and computational feasibility. Further implementation details can be found in Appendix C.

5.2 Revenue Experiments

The results of revenue experiments are presented in Table 1, where we use the notation $n \times m(X)$ to denote an auction with n

bidders and m items of setting (X). We use ‘R’ to denote randomized methods and ‘D’ for deterministic methods.

Comparison with Deterministic Baselines. Among the deterministic baselines, OD-VVCA outperforms them in most settings, especially in large-scale asymmetric scenarios. Specifically, the comparison between OD-VVCA and VCG demonstrates that integrating affine parameters into VCG significantly enhances revenue, and the comparison between OD-VVCA and Item-Myerson indicates the strong revenue performance of OD-VVCA. OD-VVCA is also distinctly advantageous over BBBVCA, which optimizes VVCA parameters under a fixed winning allocation, while the optimization of OD-VVCA explicitly considers the change in winning allocations. Additionally, OD-VVCA consistently outperforms FO-VVCA across various asymmetric settings, underscoring the efficacy of the Gaussian smoothing technique to optimize $Z_S(e^\alpha, \lambda)$.

Comparison with Randomized Baselines. Among the randomized baselines, Lottery AMA, AMenuNet, and GemNet all demonstrate commendable performance in smaller settings. However, in larger settings, GemNet fails to compute feasible allocation solutions within a reasonable time. This is due to the overly complex

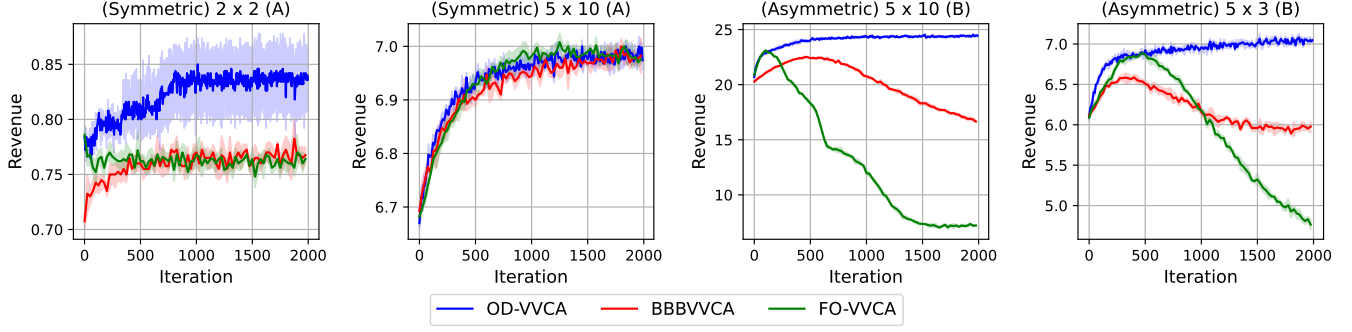


Figure 2: Training curves for OD-VVCA, BBBVCA, and FO-VVCA under 2×2 (A), 5×10 (A), 5×10 (B) and 5×3 (B). The plots depict average results across 5 runs and the 95%

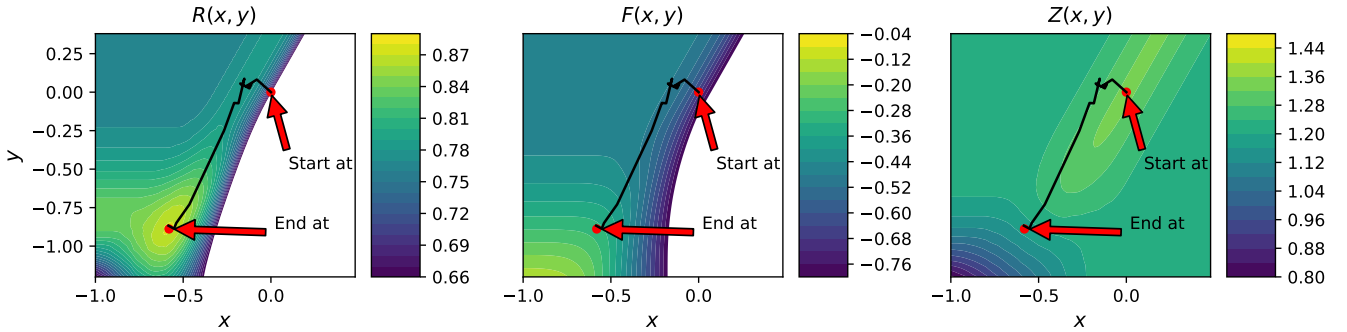


Figure 3: Training dynamics of OD-VVCA over 2000 iterations in 2×2 (A). The trajectory of (x, y) is simultaneously plotted on $R(x, y)$, $F(x, y)$, and $Z(x, y)$. The trajectory converges at a point balancing the values of both $F(x, y)$ and $Z(x, y)$.

constraints of its mixed-integer linear programming to ensure allocation feasibility, which makes it computationally expensive. For Lottery AMA and AMenuNet, as shown in Table 1, OD-VVCA consistently outperforms both of them. Lottery AMA and AMenuNet predefine a candidate size, denoted by s , and make these s candidate allocations learnable, resulting in $O(snm)$ learnable parameters. Ideally, s could be set arbitrarily large to maximize performance, but in practice, it is limited by computational resources. To balance performance with feasibility, we set s to values in $\{32, 128, 1024, 4096\}$ for both methods. These results suggest that within the constraints of computational resources, OD-VVCA is more efficient than Lottery AMA and AMenuNet, particularly in handling larger settings.

5.3 Case Study

Revenue Trajectory. To further illustrate the advantages of the proposed OD-VVCA over BBBVCA and FO-VVCA, we present the revenue trajectories of them during training in symmetric (5×10 (A)) and asymmetric (5×10 (B) and 5×3 (B)) scenarios in Figure 2. We can see that while all three methods converge to similar revenue in the symmetric 5×10 (A) scenario, the revenue of BBBVCA and FO-VVCA declines significantly in the two asymmetric settings. The superior performance of OD-VVCA over BBBVCA highlights the advantages of computing gradients with explicit consideration of changes in the winning allocation. Meanwhile, the comparison

between OD-VVCA and FO-VVCA indicates the effectiveness and stability of incorporating the Gaussian smoothing technique in optimizing VVCA parameters.

Training Process. To visualize the training process of OD-VVCA, we adopt a symmetric simplification for bidders and items in 2×2 (A). Specifically, we fix the bidder weights $w_1 = w_2 = 1$ and $\lambda_1(\emptyset) = \lambda_2(\emptyset) = 0$. For other boosts, we set

$$\begin{cases} \lambda_1(\{1\}) = \lambda_2(\{1\}) = \lambda_1(\{2\}) = \lambda_2(\{2\}) = x, \\ \lambda_1(\{1, 2\}) = \lambda_2(\{1, 2\}) = y. \end{cases}$$

Under this simplification, when fixing a dataset of valuation $\mathcal{S}$, we use $R(x, y)$, $F(x, y)$, and $Z(x, y)$ to denote $R_{\mathcal{S}}(\mathbf{w}, \lambda)$, $F_{\mathcal{S}}(\mathbf{w}, \lambda)$, $Z_{\mathcal{S}}(\mathbf{w}, \lambda)$, respectively. In Figure 3, we illustrate the trajectory of (x, y) concurrently on $R(x, y)$, $F(x, y)$, and $Z(x, y)$. Here, $R(x, y) = F(x, y) + Z(x, y)$ according to Equation (9). The trajectory initiates from a VCG auction ($x = y = 0$), and converges to an improved solution that balances the values of both $F(x, y)$ and $Z(x, y)$, which shows the efficacy of our Gaussian smoothing technique to optimize $Z(x, y)$.

Visualization of Gaussian Smoothing. To further demonstrate the impact of the Gaussian smoothing technique in OD-VVCA, we explore a 2×2 scenario (A), adjusting only one of the boost variables λ while maintaining the remaining VVCA parameters fixed. We

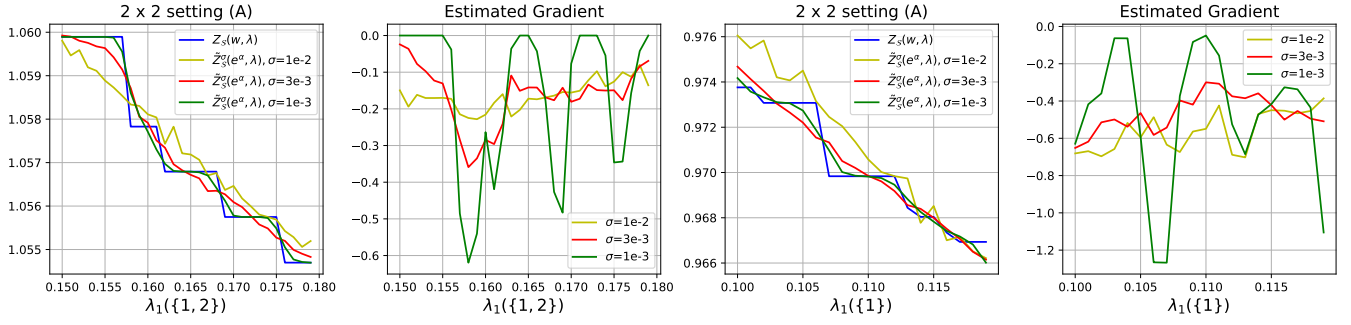


Figure 4: Visualization of $Z_S(e^\alpha, \lambda)$, $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$, and the corresponding estimated gradient $\tilde{\nabla} \tilde{Z}_S^\sigma(e^\alpha, \lambda)$ in $2 \times 2(A)$ with respect to one parameter.

Table 2: The total training time for different methods. We use the same parallelizable dynamic programming for winning allocation determination in both BBBVVCA and OD-VVCA.

	# Iterations	2×2	2×5	5×3	10×5	30×5	3×10	5×10
Lottery AMA	10000	1min40s	15min	8min	20min	6h	1h40min	9h
AMenuNet	1000	12min	22min	24min	1h20min	50min*	1h20min	3h30min
BBBVVCA	4000	1min	3min	3min	16min	1h	2h	4h20min
OD-VVCA	2000	1min20s	7min	6min	40min	3h	6h	9h

*We reduce the batch size of AMenuNet in the 30×5 scenarios due to GPU constraints.

plot $Z_S(w, \lambda)$ alongside the computed $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$ for three distinct σ values in Figure 4. The figure shows that $Z_S(w, \lambda)$ experiences non-differentiability at several points. The Gaussian smoothing technique addresses this by smoothing $Z_S(w, \lambda)$, with varying smoothness achieved by different σ values. Specifically, a smaller σ provides a closer approximation of $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$ to $Z_S(w, \lambda)$, and a larger σ results in a less precise approximation but more effectively captures the general trend with a flatter gradient. Though these insights emphasize the criticality of careful parameter selection in OD-VVCA, they do not necessarily imply difficulty in tuning the hyperparameters. In practice, our experiments indicate that σ values within $\{0.01, 0.003, 0.001\}$ are generally well-suited for most scenarios.

5.4 Training Time

We document the total training time of Lottery AMA [10], AMenuNet [11], BBBVVCA [32], and the proposed OD-VVCA for different auction scales in Table 2. We tuned the number of training iterations for each method based on our empirical observations of their convergence rates, which are also listed in the table. As indicated in the table, OD-VVCA demonstrates acceptable training times across all settings. This result, along with the similarly acceptable training times of BBBVVCA, underscores the efficiency of the parallelized dynamic programming winner determination algorithm. Furthermore, the relatively small number of iterations highlights the effective convergence of our objective decomposition method.

6 Conclusion and Future Work

In this paper, we introduce OD-VVCA, an objective decomposition approach for designing deterministic VVCAs. Our method employs a parallelizable DP algorithm to compute the winning allocation and revenue in a VVCA efficiently. Subsequently, we decompose the objective function of revenue into continuous and discontinuous components. The continuous portion can be directly optimized by taking derivatives. For the discontinuous part, we use Monte Carlo estimation to approximate the gradient of its Gaussian-smoothed version. Finally, we demonstrate the efficacy, stability, and scalability of OD-VVCA in various auction settings through experimental results. As for future work, while OD-VVCA is tailored for general combinatorial auctions, exploring its adaptation for industrial applications presents an intriguing avenue. Additionally, given that the Gaussian smoothing technique requires extra iterations to approximate the gradient via Monte Carlo sampling, it would be valuable to investigate more efficient methods for gradient approximation.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No. 62572010), Alibaba Group through the Alibaba Innovative Research Program, and Peking University-Alimama Joint Laboratory of AI Innovation. The authors thank all anonymous reviewers for their helpful feedback.

References

- [1] M-F Balcan, Siddharth Prasad, and Tuomas Sandholm. 2021. Learning within an instance for designing high-revenue combinatorial auctions. In *IJCAI Annual Conference*.
- [2] Maria-Florina Balcan, Tuomas Sandholm, and Ellen Vitercik. 2018. A general theory of sample complexity for multi-item profit maximization. In *Proceedings of the 2018 ACM Conference on Economics and Computation*. 173–174.
- [3] Maria-Florina Balcan, Tuomas Sandholm, and Ellen Vitercik. 2023. Generalization guarantees for multi-item profit maximization: Pricing, auctions, and randomized mechanisms. *Operations Research* (2023).
- [4] Maria-Florina F Balcan, Tuomas Sandholm, and Ellen Vitercik. 2016. Sample complexity of automated mechanism design. *Advances in Neural Information Processing Systems* 29 (2016).
- [5] Martin Bichler, Maximilian Fichtl, Stefan Heidekrüger, Nils Kohring, and Paul Sutterer. 2021. Learning equilibria in symmetric auction games using artificial neural networks. *Nature machine intelligence* 3, 8 (2021), 687–695.
- [6] Vincent Conitzer and Tuomas Sandholm. 2004. Self-interested automated mechanism design and implications for optimal combinatorial auctions. In *Proceedings of the 5th ACM Conference on Electronic Commerce*. 132–141.
- [7] Michael Curry, Ping-Yeh Chiang, Tom Goldstein, and John Dickerson. 2020. Certifying strategyproof auction networks. *Advances in Neural Information Processing Systems* 33 (2020), 4987–4998.
- [8] Michael Curry, Vinzenz Thoma, Darshan Chakrabarti, Stephen McAleer, Christian Kroer, Tuomas Sandholm, Niao He, and Sven Seuken. 2024. Automated design of affine maximizer mechanisms in dynamic settings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 9626–9635.
- [9] Michael J Curry, Uro Lyi, Tom Goldstein, and John P Dickerson. 2022. Learning revenue-maximizing auctions with differentiable matching. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 6062–6073.
- [10] Michael J. Curry, Tuomas Sandholm, and John P. Dickerson. 2023. Differentiable Economics for Randomized Affine Maximizer Auctions. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th–25th August 2023, Macao, SAR, China*. ijcai.org, 2633–2641. doi:10.24963/ijcai.2023/293
- [11] Zhijian Duan, Haoran Sun, Yurong Chen, and Xiaotie Deng. 2023. A Scalable Neural Network for DSIC Affine Maximizer Auction Design. *Advances in Neural Information Processing Systems* 36 (2023).
- [12] Zhijian Duan, Jingwu Tang, Yutong Yin, Zhe Feng, Xiang Yan, Manzil Zaheer, and Xiaotie Deng. 2022. A context-integrated transformer-based neural network for auction design. In *International Conference on Machine Learning*. PMLR, 5609–5626.
- [13] Paul Dütting, Zhe Feng, Harikrishna Narasimhan, David Parkes, and Sai Srivatsa Ravindranath. 2019. Optimal auctions through deep learning. In *International Conference on Machine Learning*. PMLR, 1706–1715.
- [14] Paul Dütting, Zhe Feng, Harikrishna Narasimhan, David C Parkes, and Sai Srivatsa Ravindranath. 2023. Optimal auctions through deep learning: Advances in differentiable economics. *J. ACM* (2023).
- [15] Benjamin Edelman, Michael Ostrovsky, and Michael Schwarz. 2007. Internet advertising and the generalized second-price auction: Selling billions of dollars worth of keywords. *American economic review* 97, 1 (2007), 242–259.
- [16] Noah Golowich, Harikrishna Narasimhan, and David C Parkes. 2018. Deep Learning for Multi-Facility Location Mechanism Design. In *IJCAI*. 261–267.
- [17] Dmitry Ivanov, Iskander Safulin, Igor Filippov, and Ksenia Balabaeva. 2022. Optimal-er auctions through attention. *Advances in Neural Information Processing Systems* 35 (2022), 34734–34747.
- [18] Philippe Jehiel, Moritz Meyer-Ter-Vehn, and Benny Moldovanu. 2007. Mixed bundling auctions. *Journal of Economic Theory* 134, 1 (2007), 494–512.
- [19] Ron Lavi, Ahuva Mu’Alem, and Noam Nisan. 2003. Towards a characterization of truthful combinatorial auctions. In *44th Annual IEEE Symposium on Foundations of Computer Science, 2003. Proceedings*. IEEE, 574–583.
- [20] Xuejian Li, Ze Wang, Bingqi Zhu, Fei He, Yongkang Wang, and Xingxing Wang. 2024. Deep automated mechanism design for integrating ad auction and allocation in feed. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1211–1220.
- [21] Anton Likhodedov and Tuomas Sandholm. 2004. Methods for boosting revenue in combinatorial auctions. In *AAAI*. 232–237.
- [22] Anton Likhodedov, Tuomas Sandholm, et al. 2005. Approximating revenue-maximizing combinatorial auctions. In *AAAI*, Vol. 5. 267–274.
- [23] Xiangyu Liu, Chuan Yu, Zhilin Zhang, Zhenzhe Zheng, Yu Rong, Hongtao Lv, Da Huo, Yiqing Wang, Dagui Chen, Jian Xu, et al. 2021. Neural auction: End-to-end learning of auction mechanisms for e-commerce advertising. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 3354–3364.
- [24] Roger B Myerson. 1981. Optimal auction design. *Mathematics of operations research* 6, 1 (1981), 58–73.
- [25] Neehar Peri, Michael Curry, Samuel Dooley, and John Dickerson. 2021. PreferenceNet: Encoding human preferences in auction design with deep learning. *Advances in Neural Information Processing Systems* 34 (2021), 17532–17542.
- [26] Jad Rahme, Samy Jelassi, Joan Bruna, and S Matthew Weinberg. 2021. A permutation-equivariant neural network architecture for auction design. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 5664–5672.
- [27] Jad Rahme, Samy Jelassi, and S. Matthew Weinberg. 2021. Auction Learning as a Two-Player Game. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=YHdeAO61l6T>
- [28] Sai Srivatsa Ravindranath, Zhe Feng, Shira Li, Jonathan Ma, Scott D Kominers, and David C Parkes. 2021. Deep learning for two-sided matching. *arXiv preprint arXiv:2107.03427* (2021).
- [29] Sai Srivatsa Ravindranath, Yanchen Jiang, and David C. Parkes. 2023. Data Market Design through Deep Learning. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=sgCrNMOuXp>
- [30] Kevin Roberts. 1979. The characterization of implementable choice rules. *Aggregation and revelation of preferences* 12, 2 (1979), 321–348.
- [31] Michael H Rothkopf, Aleksandar Pekeć, and Ronald M Harstad. 1998. Computationally manageable combinatorial auctions. *Management science* 44, 8 (1998), 1131–1147.
- [32] Tuomas Sandholm and Anton Likhodedov. 2015. Automated design of revenue-maximizing combinatorial auctions. *Operations Research* 63, 5 (2015), 1000–1025.
- [33] Weiran Shen, Pingzhong Tang, and Song Zuo. 2019. Automated Mechanism Design via Neural Networks. In *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems (Montreal QC, Canada) (AAMAS ’19)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 215–223.
- [34] Pingzhong Tang and Tuomas Sandholm. 2012. Mixed-bundling auctions with reserve prices. In *AAMAS*. 729–736.
- [35] William Vickrey. 1961. Counterspeculation, auctions, and competitive sealed tenders. *The Journal of finance* 16, 1 (1961), 8–37.
- [36] Tonghan Wang, Yanchen Jiang, and David C. Parkes. 2024. GemNet: Menu-Based, Strategy-Proof Multi-Bidder Auctions Through Deep Learning. In *Proceedings of the 25th ACM Conference on Economics and Computation (New Haven, CT, USA) (EC ’24)*. Association for Computing Machinery, New York, NY, USA, 1100. doi:10.1145/3670865.3673454

A Time and Space Complexity Analysis of the Winning Allocation Deterministic Dynamic Programming

Proposition A.1. *The time and space complexity of the winning allocation deterministic dynamic programming is $O(n3^m)$ and $O(n2^m)$, respectively.*

PROOF. The initialization loop and the final loop involve 2^m and n enumerations, respectively. In the DP loop, for each bidder $i \in [2, n]$ and bundle $|S| = k$, we enumerate all 2^k subsets of S . Consequently, the total number of enumerations for bidder i is given by

$$\sum_{k=0}^m \binom{m}{k} 2^k = (2+1)^m = 3^m.$$

Since the computation during each enumeration is $O(1)$, the overall time complexity is $O(2^m) + O((n-1)3^m) + O(n) = O(n3^m)$.

Regarding space complexity, apart from the $O(n2^m)$ needed for input valuations and VVCA parameters, we require an additional $O(n2^m)$ space to store MAW and AB. Notably, there is no need to store Int. Therefore, the total space complexity is $O(n2^m)$. $\square$

B Proof of Proposition 4.1

Proposition 4.1. *$\tilde{Z}_S^\sigma(e^\alpha, \lambda)$ is continuous and differentiable with derivatives*

$$\nabla_\alpha \tilde{Z}_S^\sigma(e^\alpha, \lambda) = \mathbb{E}_{\epsilon \sim N(0,1)^n, \delta \sim N(0,1)^{n \times 2^m}} \frac{\Delta_\epsilon^\delta}{\sigma} \epsilon,$$

$$\nabla_\lambda \tilde{Z}_S^\sigma(e^\alpha, \lambda) = \mathbb{E}_{\epsilon \sim N(0,1)^n, \delta \sim N(0,1)^{n \times 2^m}} \frac{\Delta_\epsilon^\delta}{\sigma} \delta,$$

where we define $\Delta_\epsilon^\delta := Z_S(e^{\alpha+\sigma\epsilon}, \lambda + \sigma\delta) - Z_S(e^\alpha, \lambda)$.

To prove Proposition 4.1, we first present a useful lemma:

Lemma B.1. *Let $W : \mathbb{R}^n \rightarrow \mathbb{R}$ be a bounded function such that $\max_{x \in \mathbb{R}^n} |W(x)| \leq M_W$, and denote $\tilde{W}^\sigma := \mathbb{E}_{u \sim N(0,1)^n} [W(x + \sigma u)]$ with $\sigma > 0$ as the Gaussian smoothing approximation of W . Then $\tilde{W}^\sigma$ is differentiable.*

PROOF. For any $x = (x_1, x_2, \dots, x_n)$, without loss of generality, let's examine the differentiability of $\tilde{W}^\sigma(x)$ with respect to x_1 . We define $x_{-1} = (x_2, x_3, \dots, x_n)$, and for any $u \in \mathbb{R}^n$, we denote $u_{-1} = (u_1, u_{-1})$.

Existence of $\frac{\partial \tilde{W}^\sigma(x)}{\partial x_1}$. We prove the existence of $\frac{\partial \tilde{W}^\sigma(x)}{\partial x_1}$ by deriving according to the definition of partial derivative:

$$\begin{aligned} & \lim_{\Delta x_1 \rightarrow 0} \frac{\tilde{W}^\sigma(x_1 + \Delta x_1, x_{-1}) - \tilde{W}^\sigma(x_1, x_{-1})}{\Delta x_1} \\ &= \lim_{\Delta x_1 \rightarrow 0} \frac{1}{\Delta x_1} \mathbb{E}_{u \sim N(0, I_n)} [W(x_1 + \Delta x_1 + \sigma u_1, x_{-1} + \sigma u_{-1})] \\ &\quad - \frac{1}{\Delta x_1} \mathbb{E}_{u \sim N(0, I_n)} [W(x + \sigma u)] \\ &= \lim_{\Delta x_1 \rightarrow 0} \frac{1}{\Delta x_1} \mathbb{E}_{u' := (u'_1, u_{-1}) \sim N((\frac{\Delta x_1}{\sigma}, 0_{n-1}), I_n)} [W(x + \sigma u')] \\ &\quad - \frac{1}{\Delta x_1} \mathbb{E}_{u \sim N(0, I_n)} [W(x + \sigma u)] \\ &= \lim_{\Delta x_1 \rightarrow 0} \left(\frac{1}{2\pi} \right)^{\frac{n}{2}} \int W(x + \sigma u) \frac{1}{\Delta x_1} \\ &\quad \cdot \left(\exp \left(-\frac{1}{2} \left\| (u_1 - \frac{\Delta x_1}{\sigma}, u_{-1}) \right\|^2 \right) - \exp \left(-\frac{1}{2} \|u\|^2 \right) \right) du \\ &\stackrel{(a)}{=} \left(\frac{1}{2\pi} \right)^{\frac{n}{2}} \int W(x + \sigma u) \frac{u_1}{\sigma} \exp \left(-\frac{1}{2} \|u\|^2 \right) du \\ &= \mathbb{E}_{u \sim N(0, I)} \left[\frac{W(x + \sigma u)}{\sigma} u_1 \right] \\ &\stackrel{(b)}{=} \mathbb{E}_{u \sim N(0, I)} \left[\frac{W(x + \sigma u) - W(x)}{\sigma} u_1 \right], \end{aligned} \tag{10}$$

where (b) holds since $\mathbb{E}_{u \sim N(0, I)} [W(x)u_1] = 0$, and (a) in Equation (10) essentially comes from Taylor expansion: We first define $f(t) := \exp(-\frac{1}{2}\|(t, u_{-1})\|^2)$, then we use Taylor expansion of $f(u_1 - \frac{\Delta x_1}{\sigma})$ at u_1 with Lagrange remainder, that is,

$$\begin{aligned} f(u_1 - \frac{\Delta x_1}{\sigma}) &= f(u_1) + f'(u_1) \left(-\frac{\Delta x_1}{\sigma} \right) + f''(\xi(\Delta x_1, u_1)) \frac{(\Delta x_1)^2}{\sigma^2} \\ &= f(u_1) + u_1 \exp \left(-\frac{1}{2} \|(u_1, u_{-1})\|^2 \right) \frac{\Delta x_1}{\sigma} \\ &\quad + (\xi^2(\Delta x_1, u_1) - 1) \exp \left(-\frac{1}{2} \|(\xi(\Delta x_1, u_1), u_{-1})\|^2 \right) \frac{(\Delta x_1)^2}{\sigma^2} \end{aligned}$$

where $\xi(\Delta x_1, u_1)$ is between $u_1 - \frac{\Delta x_1}{\sigma}$ and u_1 . On top of that, (a) is derived by

$$\begin{aligned} & \lim_{\Delta x_1 \rightarrow 0} \int W(x + \sigma u) \frac{1}{\Delta x_1} \cdot \left(\exp \left(-\frac{1}{2} \left\| (u_1 - \frac{\Delta x_1}{\sigma}, u_{-1}) \right\|^2 \right) \right. \\ &\quad \left. - \exp \left(-\frac{1}{2} \|(u_1, u_{-1})\|^2 \right) \right) du \\ &= \lim_{\Delta x_1 \rightarrow 0} \int W(x + \sigma u) du \frac{1}{\Delta x_1} \cdot \left(u_1 \exp \left(-\frac{1}{2} \|(u_1, u_{-1})\|^2 \right) \frac{\Delta x_1}{\sigma} \right. \\ &\quad \left. + (\xi^2(\Delta x_1, u_1) - 1) \exp \left(-\frac{1}{2} \|(\xi(\Delta x_1, u_1), u_{-1})\|^2 \right) \frac{(\Delta x_1)^2}{\sigma^2} \right) \\ &= \lim_{\Delta x_1 \rightarrow 0} \int W(x + \sigma u) du \cdot \left(\frac{u_1}{\sigma} \exp \left(-\frac{1}{2} \|u\|^2 \right) \right. \\ &\quad \left. + (\xi^2(\Delta x_1, u_1) - 1) \exp \left(-\frac{1}{2} \|(\xi(\Delta x_1, u_1), u_{-1})\|^2 \right) \frac{\Delta x_1}{\sigma^2} \right) \\ &\stackrel{(c)}{=} \lim_{\Delta x_1 \rightarrow 0} \int W(x + \sigma u) \frac{u_1}{\sigma} \exp \left(-\frac{1}{2} \|u\|^2 \right) du \\ &= \int W(x + \sigma u) \frac{u_1}{\sigma} \exp \left(-\frac{1}{2} \|u\|^2 \right) du, \end{aligned} \tag{11}$$

where (c) is equivalent to the truth of the following equation:

$$\begin{aligned}
& \lim_{\Delta x_1 \rightarrow 0} \left| \Delta x_1 \int W(x + \sigma u) \cdot (\xi^2(\Delta x_1, u_1) - 1) \right. \\
& \quad \cdot \exp\left(-\frac{1}{2} \|(\xi(\Delta x_1, u_1), \mathbf{u}_{-1})\|^2\right) du \Big| \\
& \leq \lim_{\Delta x_1 \rightarrow 0} |\Delta x_1| \cdot |W(x + \sigma u)| \\
& \quad \cdot \left| \int (\xi^2(\Delta x_1, u_1) - 1) \exp\left(-\frac{1}{2} \|(\xi(\Delta x_1, u_1), \mathbf{u}_{-1})\|^2\right) du \right| \quad (12) \\
& \leq \lim_{\Delta x_1 \rightarrow 0} |\Delta x_1| \cdot M_W \\
& \quad \cdot \left| \int (\xi^2(\Delta x_1, u_1) - 1) \exp\left(-\frac{1}{2} \|(\xi(\Delta x_1, u_1), \mathbf{u}_{-1})\|^2\right) du \right| \\
& = \lim_{\Delta x_1 \rightarrow 0} |\Delta x_1| \cdot M_W \cdot \left| \int K(\xi(\Delta x_1, u_1), \mathbf{u}_{-1}) du \right| \\
& \leq \lim_{\Delta x_1 \rightarrow 0} |\Delta x_1| \cdot M_W \cdot \int |K(\xi(\Delta x_1, u_1), \mathbf{u}_{-1})| du
\end{aligned}$$

where we define $K(s, \mathbf{u}_{-1}) := (s^2 - 1) \exp(-\frac{1}{2} \|(s, \mathbf{u}_{-1})\|^2)$ with $|s - u_1| \leq \left|\frac{\Delta x_1}{\sigma}\right|$, and we prove (c) by showing that when $|\Delta x_1| < \sigma$, the term

$$\int |K(\xi(\Delta x_1, u_1), \mathbf{u}_{-1})| du \leq \int_{s \in [u_1 - \left|\frac{\Delta x_1}{\sigma}\right|, u_1 + \left|\frac{\Delta x_1}{\sigma}\right|]} |K(s, \mathbf{u}_{-1})| du \quad (13)$$

is bounded: Given the definition of K , there exists a constant $C_K > 1$ such that when $s > C_K$, $|K(s, \mathbf{u}_{-1})|$ is monotone decreasing with respect to s , and when $s < -C_K$, $|K(s, \mathbf{u}_{-1})|$ is monotone increasing with respect to s . Based on that, we discuss the following three cases of the range of u_1 :

- (1) When $u_1 > 2C_K$, then $s \geq u - \left|\frac{\Delta x_1}{\sigma}\right| > 2C_K - 1 > C_K$, so that $|K(s, \mathbf{u}_{-1})|$ is monotone decreasing with respect to s . Therefore we have

$$\begin{aligned}
& \int_{u_1 > 2C_K} \max_{s \in [u_1 - \left|\frac{\Delta x_1}{\sigma}\right|, u_1 + \left|\frac{\Delta x_1}{\sigma}\right|]} |K(s, \mathbf{u}_{-1})| du \\
& \leq \int_{u_1 > 2C_K} \left| K(u_1 - \left|\frac{\Delta x_1}{\sigma}\right|, \mathbf{u}_{-1}) \right| du \\
& \leq \int_{u_1 > 2C_K} |K(u_1 - 1, \mathbf{u}_{-1})| du =: C_1,
\end{aligned}$$

where C_1 is a constant independent of $|\Delta x_1|$.

- (2) When $-2C_K \leq u_1 \leq 2C_K$, we have

$$\begin{aligned}
& \int_{-2C_K \leq u_1 \leq 2C_K} \max_{s \in [u_1 - \left|\frac{\Delta x_1}{\sigma}\right|, u_1 + \left|\frac{\Delta x_1}{\sigma}\right|]} |K(s, \mathbf{u}_{-1})| du \\
& \leq \int_{-2C_K \leq u_1 \leq 2C_K} \max_{-2C_K \leq u'_1 \leq 2C_K} \left(\max_{s \in [u'_1 - \left|\frac{\Delta x_1}{\sigma}\right|, u'_1 + \left|\frac{\Delta x_1}{\sigma}\right|]} |K(s, \mathbf{u}_{-1})| \right) du \\
& \leq \int_{-2C_K \leq u_1 \leq 2C_K} \max_{-(2C_K+1) \leq s \leq (2C_K+1)} |K(s, \mathbf{u}_{-1})| du =: C_2
\end{aligned}$$

where C_2 is a constant independent of $|\Delta x_1|$.

- (3) When $u_1 < -2C_K$, then $s \leq u + \left|\frac{\Delta x_1}{\sigma}\right| < -2C_K + 1 < C_K$, so that $|K(s, \mathbf{u}_{-1})|$ is monotone increasing with respect to s . Similar to case 1, we have

$$\begin{aligned}
& \int_{u_1 < -2C_K} \max_{s \in [u_1 - \left|\frac{\Delta x_1}{\sigma}\right|, u_1 + \left|\frac{\Delta x_1}{\sigma}\right|]} |K(s, \mathbf{u}_{-1})| du \\
& \leq \int_{u_1 < -2C_K} \left| K(u_1 + \left|\frac{\Delta x_1}{\sigma}\right|, \mathbf{u}_{-1}) \right| du \\
& \leq \int_{u_1 < -2C_K} |K(u_1 + 1, \mathbf{u}_{-1})| du =: C_3
\end{aligned}$$

where C_3 is a constant independent of $|\Delta x_1|$.

Therefore, Equation (13) is bounded by $C_1 + C_2 + C_3$ which is independent of $|\Delta x_1|$, and the proof of Equation (12) is complete. Subsequently, the proof of (b) in Equation (11) is established. Following that, the proof of (a) in Equation (10) is concluded. Consequently, the existence of $\frac{\partial \tilde{W}^\sigma(\mathbf{x})}{\partial x_1}$ is derived from Equation (10).

Continuity of $\frac{\partial \tilde{W}^\sigma(\mathbf{x})}{\partial x_1}$. Given σ , let $Q(\mathbf{x}) := W(\frac{\mathbf{x}}{\sigma})$ and $H(\mathbf{u}) := -u_1 \exp(-\frac{1}{2} \|\mathbf{u}\|^2)$, according to Equation (10) we have

$$\begin{aligned}
\frac{\partial \tilde{W}^\sigma(\mathbf{x})}{\partial x_1} &= \frac{1}{\sigma} \left(\frac{1}{2\pi} \right)^{\frac{n}{2}} \int W(\mathbf{x} + \sigma \mathbf{u}) u_1 \exp\left(-\frac{1}{2} \|\mathbf{u}\|^2\right) du \\
&= \frac{1}{\sigma} \left(\frac{1}{2\pi} \right)^{\frac{n}{2}} \int Q(\mathbf{u} + \frac{\mathbf{x}}{\sigma}) H(-\mathbf{u}) du \\
&= \frac{1}{\sigma} \left(\frac{1}{2\pi} \right)^{\frac{n}{2}} \int Q(\mathbf{u}) H(\frac{\mathbf{x}}{\sigma} - \mathbf{u}) du \\
&=: \frac{1}{\sigma} \left(\frac{1}{2\pi} \right)^{\frac{n}{2}} C(\frac{\mathbf{x}}{\sigma}),
\end{aligned}$$

where we can see that $C(\frac{\mathbf{x}}{\sigma})$ is a convolution function. As for the continuity of $C(\frac{\mathbf{x}}{\sigma})$, we have:

$$\begin{aligned}
& \lim_{h \rightarrow 0} \left| C(\frac{\mathbf{x}}{\sigma} + \mathbf{h}) - C(\frac{\mathbf{x}}{\sigma}) \right| \\
&= \lim_{h \rightarrow 0} \left| \int H(\frac{\mathbf{x}}{\sigma} + \mathbf{h} - \mathbf{u}) Q(\mathbf{u}) du - \int H(\frac{\mathbf{x}}{\sigma} - \mathbf{u}) Q(\mathbf{u}) du \right| \\
&\leq \lim_{h \rightarrow 0} \int \left| H(\frac{\mathbf{x}}{\sigma} + \mathbf{h} - \mathbf{u}) - H(\frac{\mathbf{x}}{\sigma} - \mathbf{u}) \right| |Q(\mathbf{u})| du \\
&\stackrel{(a)}{\leq} \lim_{h \rightarrow 0} M_W \int \left| H(\frac{\mathbf{x}}{\sigma} + \mathbf{h} - \mathbf{u}) - H(\frac{\mathbf{x}}{\sigma} - \mathbf{u}) \right| du \\
&\stackrel{(b)}{=} 0,
\end{aligned}$$

where (a) holds because $Q_\sigma(\mathbf{x}) := W(\frac{\mathbf{x}}{\sigma})$ is bounded by $M_W := \max_{\mathbf{x}} W(\mathbf{x})$, and (b) holds because $H(\mathbf{u})$ is integrable. As a result, $C(\frac{\mathbf{x}}{\sigma})$ is continuous with respect to $\mathbf{x}$, and then $\frac{\partial \tilde{W}^\sigma(\mathbf{x})}{\partial x_1}$ is also continuous with respect to x_1 .

Differentiability of $\tilde{W}^\sigma$. From the preceding discussion, it is evident that $\tilde{W}^\sigma(\mathbf{x})$ is differentiable with respect to x_1 . This technique can be extended straightforwardly to $x_2, x_3, \dots, x_n$. Therefore, $\tilde{W}^\sigma(\mathbf{x})$ is differentiable (and thus continuous) with respect to $\mathbf{x}$. $\square$

Table 3: Hyperparameters of OD-VVCA and FO-VVCA.

(a) Small-scale settings.

Hyperparameter	Symmetric					Asymmetric				
	2×2(A)	2×5(A)	3×10(A)	2×2(D)	3×10(D)	5×3(B)	3×10(B)	2×5(C)	5×3(C)	3×10(C)
Learning Rate	0.01	0.001	0.001	0.01	0.001	0.001	0.001	0.001	0.001	0.001
n_r	8	8	8	8	8	8	8	8	8	8
σ	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01
Iteration	2000	2000	2000	2000	2000	2000	2000	2000	2000	2000
Batch Size	1024	2048	1024	1024	1024	1024	1024	2048	1024	1024

(b) Large-scale settings.

Hyperparameter	Symmetric				Asymmetric					
	5×10(A)	10×5(A)	5×10(D)	10×5(D)	5×10(B)	10×5(B)	30×5(B)	5×10(C)	10×5(C)	30×5(C)
Learning Rate	0.0003	0.0003	0.0003	0.0003	0.005	0.005	0.005	0.005	0.005	0.005
n_r	8	8	8	8	8	8	8	8	8	8
σ	0.001	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01
Iteration	2000	2000	2000	2000	2000	2000	2000	2000	2000	2000
Batch Size	1024	1024	1024	1024	1024	1024	1024	1024	1024	1024

Table 4: Candidate size of Lottery AMA and AMenuNet in different settings.

(a) Small-scale settings.

Method	Symmetric					Asymmetric				
	2×2(A)	2×5(A)	3×10(A)	2×2(D)	3×10(D)	5×3(B)	3×10(B)	2×5(C)	5×3(C)	3×10(C)
Lottery AMA	32	128	1024	32	1024	128	1024	128	128	1024
AMenuNet	32	128	1024	32	1024	128	1024	128	128	1024

(b) Large-scale settings.

Method	Symmetric				Asymmetric					
	5×10(A)	10×5(A)	5×10(D)	10×5(D)	5×10(B)	10×5(B)	30×5(B)	5×10(C)	10×5(C)	30×5(C)
Lottery AMA	4096	4096	4096	4096	4096	4096	4096	4096	4096	4096
AMenuNet	4096	4096	4096	4096	4096	4096	4096	4096	4096	4096

PROOF OF PROPOSITION 4.1. $\tilde{Z}_S^\sigma(e^\alpha, \lambda)$ is defined as the Gaussian smoothing approximation of $Z_S(e^\alpha, \lambda)$, whose range is $[\min_{A \in \mathcal{A}} \sum_{i=1}^n v_i(A), \max_{V \in \mathcal{S}} \max_{A \in \mathcal{A}} \sum_{i=1}^n v_i(A)]$. Therefore, the proof is done by applying Lemma B.1. $\square$

C Further Implementation Details

For OD-VVCA, we select the number of random directions n_r from $\{8, 16, 32\}$ and the standard deviation σ from $\{0.01, 0.003, 0.001\}$. The training starts at classic VCG, i.e., $\mathbf{w} = 1$ and $\lambda = 0$. We use

the same hyperparameters for OD-VVCA and the ablation version FO-VVCA in all the settings, and we list the main hyperparameters in Table 3.

For Lottery AMA [10] and AMenuNet [11], we adopt the same hyperparameters as those specified in their original publications, with the exception of the candidate size. Due to computational resource limitations, the size of the candidate allocation set cannot be excessively large. Therefore, we set the candidate size based on the complexity of the auction scenario to make a balance between performance and computational feasibility, as detailed in Table 4.